

Beyond Task-Only Matching: Personalized Skill Routing with Counterfactual Evaluation

Tianle Wang^{2*}, Yanghe Zou^{1*}, Xiang Liu^{1,3}, Ziyao Huang⁴, Chenchen Fu¹, Weiwei Wu¹

¹School of Computer Science and Engineering, Southeast University

²College of Software Engineering, Southeast University

³Department of Computer Science and Engineering, The Chinese University of Hong Kong

⁴Department of Computer Science, City University of Hong Kong

Abstract

The rapid expansion of reusable skill repositories makes skill routing a critical capability for large language model (LLM) agents. Existing methods treat routing as task-only semantic matching. However, when users with incompatible constraints issue an identical request, this assumption conflates task relevance with skill suitability: a task-only router can select a semantically plausible skill that is unsuitable for the requesting user. To expose this failure mode, we formulate *personalized skill routing* as profile-conditioned retrieval, in which relevance depends jointly on the task and the user profile. We first introduce a profile-counterfactual benchmark, in which the task is held fixed while changes in the user profile induce changes in the reference skill. We further construct paired counterfactual supervision and propose SkillFeed, a progressive retrieve-and-rerank framework that first establishes task-skill alignment and then learns profile-conditioned discrimination. By retrieving body-level evidence and reranking semantically similar but profile-conflicting candidates, SkillFeed identifies skills that satisfy both task requirements and user constraints. On SkillFeed-Bench, SkillFeed attains 75.1% top-1 retrieval accuracy, a 23.1-point improvement over the corresponding pretrained routing baseline. Adding profile conditioning yields a 35.1-point gain on queries where user profile changes the reference skill. This contrast shows that user profiles are most consequential precisely when they change skill suitability. Our website is publicly available at <http://www.aiskillfeed.com>.

1 Introduction

Large language model (LLM) agents increasingly reason, call tools, and execute multi-step workflows (Yao et al. 2023; Schick et al. 2024; Shen et al. 2023). In these systems, *skills* package reusable procedures, tool-use strategies, and execution constraints in versioned bundles with `SKILL.md` manifests (OpenAI 2026; Zheng et al. 2026). As public repositories grow, an agent must route each request to the skill that best supports its intended execution, making skill routing a central retrieval problem (Li, Tagkopoulous, and Tagkopoulous 2026; Zheng et al. 2026).

Most existing methods cast skill routing as task-to-skill semantic matching (Zheng et al. 2026; Li, Tagkopoulous, and Tagkopoulous 2026; Wang et al. 2026). This formulation assumes that the task uniquely determines the appropriate skill.

*These authors contributed equally.

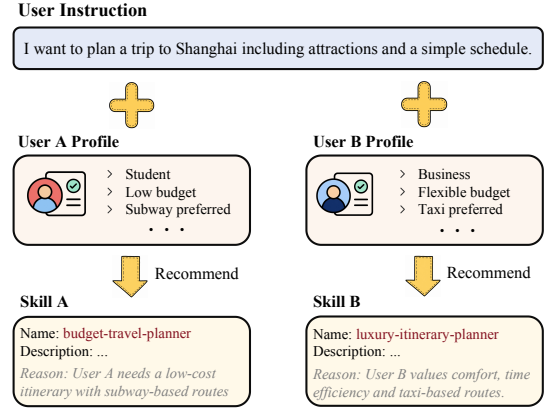


Figure 1: Motivation example of personalized skill recommendation. The same task, "planning a trip to Shanghai," is routed to a budget travel planner for a student and to a luxury itinerary planner for a business traveler. The target skill changes with user profile.

In practice, however, a task describes *what* a user wants to accomplish, whereas a user profile can determine *how* that task should be carried out. When two users issue the same request under incompatible constraints, a task-only router may select a skill that is semantically relevant but unsuitable for one of them. For example, a workflow that relies on an unavailable platform, exceeds a budget, or assumes expertise the user does not have. As illustrated in Figure 1, the same travel-planning request calls for a budget itinerary skill for a student and a premium itinerary skill for a business traveler. The task is unchanged, but the appropriate skill changes with profile attributes such as budget, payment preference, platform ecosystem, and professional background.

This failure mode cannot be established with conventional task-level evaluation. Prior benchmarks for skill learning, execution, and routing pair a request with suitable skills, while existing retrieve-and-rerank systems are trained and evaluated primarily on task-skill relevance (Li et al. 2026; Zheng et al. 2026; Wang et al. 2026). Such evaluation cannot distinguish a system that truly uses a profile from one that ignores it: when the task remains fixed, it never asks whether the predicted skill should change as the user profile changes. The

resulting data also provide limited supervision for separating skills that are equally relevant to the task but conflict in their compatibility with user constraints.

We therefore formulate *personalized skill routing* as profile-conditioned retrieval, in which skill relevance is jointly determined by a task and its task-relevant user profile. This setting raises three coupled requirements: (1) a benchmark must isolate profile-induced changes in the reference skill; (2) training data must teach the model to discriminate among profile-conflicting alternatives; (3) and the routing system must scale to large repositories while using the fine-grained applicability conditions encoded in skill bodies. We meet these requirements by constructing controlled profile-counterfactual instances that hold the task fixed while varying the user profile and reference skill. These instances both evaluate whether routing responds to profile shifts and provide supervision for profile-conditioned retrieval and reranking.

Building on this formulation, we introduce SkillFeed-Bench and SkillFeed retrieval framework. SkillFeed-Bench evaluates profile-conditioned routing over a 228K-skill repository. SkillFeed first establishes task-skill alignment and then learns profile-conditioned discrimination; at inference time, it combines broad candidate recall with test-level evidence and profile-aware reranking. This design separates the need to preserve candidate coverage at repository scale from the need to resolve profile-specific suitability among semantically similar skills. Our contributions are as follows:

- We formulate personalized skill routing as retrieval conditioned jointly on a task and a user profile. We introduce SkillFeed-Bench, a benchmark over 228K candidate skills with 329 annotated instances, including counterfactual profile-shift cases in which an unchanged task has different reference skills under different user profiles.
- We construct task-centric and counterfactual profile-conditioned training instances with semantically similar, profile-conflicting hard negatives. This supervision progressively adapts a dense retriever and trains a listwise reranker to recognize when a profile changes the suitable skill.
- We develop SkillFeed, a two-stage framework that combines dense and lexical candidate retrieval, profile-conditioned retrieval over skill chunks, reciprocal-rank fusion, and profile-aware reranking. Experiments on SkillFeed-Bench show that SkillFeed achieves 75.1% Hit@1, outperforming the pretrained baseline by 23.1%. Profile conditioning contributes a 18.6-point gain on profile-sensitive queries, and 35.1-points on profile-counterfactual samples.

2 Related Work

Personalized Retrieval and Recommendation. Personalized ranking models relevance as user-dependent, learning from interaction histories (Koren, Bell, and Volinsky 2009; Rendle et al. 2009), user profiles as conditioning input for LLM-based recommenders (Deldjoo et al. 2024; Bang and Song 2025). Personalized search and LLM agents further integrate user memory and intent into ranking and long-term interaction (Zhou et al. 2024; Westhäüßer, Minker, and Zepf

2025). These methods establish the importance of conditioning ranking on user information, but their target is an item, document, or content preference rather than an executable procedure selected for a concrete task. In skill routing, a profile contains task-relevant constraints, such as language, platform access, budget, and expertise, that can make one workflow suitable and another inapplicable. Our focus is therefore not general personalization, but the profile-induced change in the suitable skill under a fixed task, which requires supervision and evaluation instances in which changing the profile can change the reference target.

Skill Benchmarks. Recent benchmarks evaluate complementary aspects of agent skills, including skill-enabled execution (Li et al. 2026; Han et al. 2026), lifelong skill discovery (Zhang et al. 2026), continual learning and skill generation (Zhong et al. 2026; Zhou et al. 2026), and large-scale skill retrieval (Cho, Kang, and Kim 2026; Wang et al. 2026). These resources provide valuable task-level supervision and evaluation, but they do not explicitly test whether a router should revise its selected skill when an unchanged task is paired with a different user profile. In contrast, SkillFeed-Bench includes controlled profile-shift instances in which the task is fixed while the annotated reference skill changes. This design separates profile sensitivity from ordinary task-skill semantic matching.

Skill Retrieval and Routing. Skill routing builds on sparse and dense retrieval methods, including BM25 and embedding-based retrieval (Robertson and Zaragoza 2009; Wang et al. 2022; Chen et al. 2024). At repository scale, SkillRouter uses a retrieve-and-rerank pipeline, while R3-Skill studies query-conditioned routing with a two-stage retriever (Zheng et al. 2026; Wang et al. 2026); related work also explores retrieval augmentation for agentic AI (Su et al. 2026). These systems motivate efficient candidate generation and fine-grained reranking, but task relevance alone cannot distinguish skills that are semantically similar yet incompatible with different user constraints. Moreover, skill applicability may be expressed in the procedural body of a `SKILL.md` file through usage conditions, dependencies, and platform assumptions (OpenAI 2026; Zheng et al. 2026). SkillFeed extends the retrieve-and-rerank paradigm by conditioning both candidate retrieval and final discrimination on the task-relevant user profile, using body-level evidence to resolve profile-conflicting alternatives.

3 Problem Formulation and Benchmark

This section formalizes *personalized skill routing*, a setting in which the correct skill is determined jointly by the task and the user profile rather than by task semantics alone. Under a fixed task, changing user profile constraints may change the annotated reference skill, which makes conventional task-only evaluation insufficient. Therefore, we instantiate this setting with SkillFeed-Bench, a benchmark built over 228,432 community-sourced skills and 329 evaluation queries spanning ten domains, each paired with a structured user profile and an annotated reference skill.

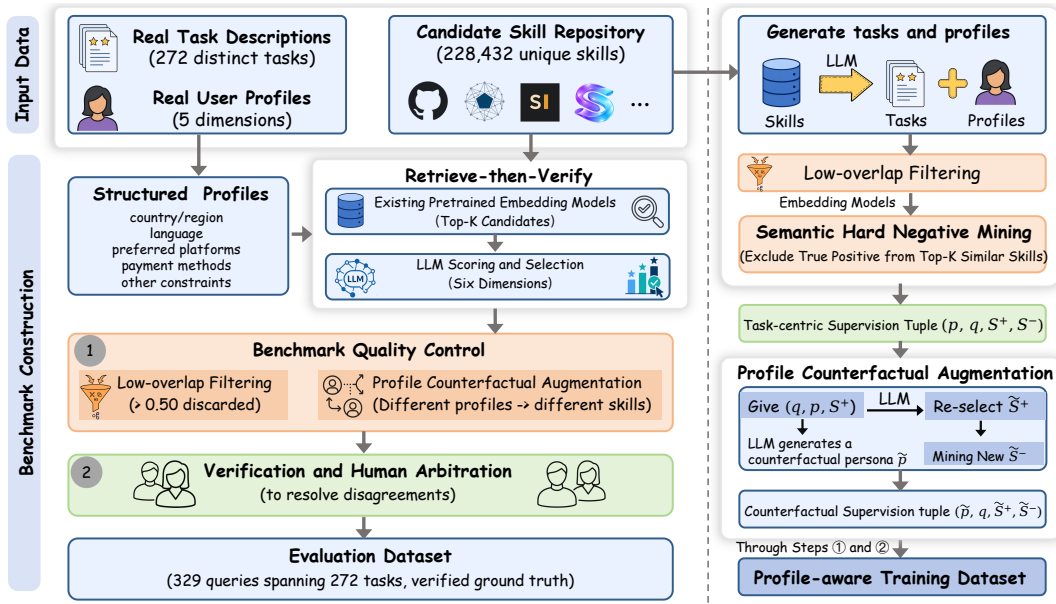


Figure 2: Overview of the dataset construction pipeline.

3.1 Personalized Skill Routing Formulation

Let q denote a natural-language task query, p a structured user profile, and $\mathcal{S} = \{S_1, \dots, S_N\}$ a repository of candidate skills. The profile contains only task-relevant contextual attributes, such as country or region, language, platform access, payment method, expertise, and other constraints. Each candidate skill $S_i = (n_i, d_i, b_i)$ comprises a name n_i , a concise description d_i , and a procedural body b_i that specifies its workflow and applicability conditions.

Personalized skill routing selects the candidate that best satisfies the task and profile jointly:

$$S_{q,p}^* = \arg \max_{S_i \in \mathcal{S}} f(q, p, S_i), \quad (1)$$

where $f(q, p, S_i)$ measures the compatibility of S_i with both q and p . This differs from task-only routing: a skill can be semantically relevant to q yet unsuitable because it conflicts with the profile’s constraints.

For scalable retrieval, the system ranks all candidates by this score and returns the top- K set:

$$\mathcal{R}_K(q, p) = \text{TopK}_{S_i \in \mathcal{S}} f(q, p, S_i), \quad (2)$$

where $\mathcal{R}_K(q, p)$ is the ordered set of the K highest-ranked skills for (q, p) . Because each evaluation instance has one annotated reference skill, candidate recall succeeds when $S_{q,p}^* \in \mathcal{R}_K(q, p)$.

3.2 SkillFeed-Benchmark Construction

Data Sources. We aggregate skills from public sources, including GitHub, SkillNet (Liang et al. 2026), and SkillSMP (SkillsMP 2026). After deduplication and format normalization, the repository contains 228,432 skills, each with a name, description, and procedural body.

Task and Profile Collection. We collect task descriptions from real LLM-agent interactions in which users sought help with specific goals. After manual screening for self-containedness, sufficient detail, and practical relevance, we retain 272 unique tasks. Each task is paired with a structured user profile covering country or region, language, preferred platforms, payment methods, and additional constraints such as regulatory requirements, budget limits, or domain-specific workflows. The profiles are designed to be concise while retaining attributes that can affect skill suitability.

Reference Target Annotation. Exhaustive annotation over 228,432 skills is impractical. We therefore use a two-stage annotation procedure. First, pretrained embedding models retrieve a top- K set of semantically relevant candidates for each task–profile pair. Second, an LLM assesses these candidates using task relevance, profile compatibility, capability match, constraint satisfaction, specificity, and overfit risk, and then performs pairwise comparisons to select the reference skill. Each annotated instance is a triplet (p, q, S) consisting of a profile, task query, and selected skill.

User Profile Counterfactual Augmentation. For an ordinary triplet (p, q, S) , the effects of the task and profile are coupled. We therefore construct counterfactual instances that hold q fixed while varying p . Given an original instance, we prompt an LLM to generate a plausible alternative profile $\tilde{p}$ that deviates from the original p and reapply the reference-target annotation procedure to $(\tilde{p}, q)$. We retain the resulting triplet $(\tilde{p}, q, \tilde{S})$ only when $\tilde{S} \neq S$. These retained pairs directly test whether a routing method changes its prediction when a profile change alters the suitable skill.

Quality Control. We apply quality controls to ensure that SkillFeed-Bench tests profile-conditioned routing rather than superficial task level semantic matching. We discard samples

with confidence scores below 0.80 and task-skill pairs with lexical overlap above 0.50, reducing trivial keyword matches and evident annotation errors. We then apply consistency checks and manually review retained instances. At least two professional human annotators with prior experience in evaluating AI agents, software tools, or programming-related tasks independently evaluate every retained original and counterfactual instance; expert arbitration resolves disagreements, and persistently ambiguous cases are excluded. These procedures reduce the risk that benchmark performance reflects spurious lexical cues or annotation artifacts.

The resulting benchmark contains 329 evaluation queries over 228K candidate skills, including 162 profile-sensitive queries (77 profile-counterfactual augmentation) and 167 profile-insensitive queries. Sensitive and Insensitive subsets are defined by presence of user context constraints. Its controlled profile shifts expose a routing problem that task-only methods cannot resolve: a system must revise its selected skill when the user profile changes the suitable execution path, while still retrieving efficiently. SkillFeed-Bench is therefore architecture-agnostic, but it demands balancing broad recall for its vast pool with fine-grained discrimination for user profile shifts. We next present SkillFeed, a two-stage retrieval-and-reranking approach designed to meet these requirements and evaluated on this benchmark.

4 Method: SkillFeed

SkillFeed is designed around three requirements that distinguish profile-conditioned routing from conventional task-skill retrieval. First, a router must retain broad functional coverage over a large repository: profile information should refine task relevance rather than replace it. Second, the decisive profile constraint often separates skills that are already semantically similar, so the model needs supervision in which a fixed task maps to different skills under different profiles. Third, these constraints frequently occur in the procedural body of a skill rather than in its short description. As illustrated in Figure 3, SkillFeed addresses these requirements jointly: it constructs counterfactual profile supervision, learns task alignment before profile-sensitive discrimination, and retrieves body-level evidence before making a final profile-conditioned ranking. Thus, the components are not interchangeable retrieval add-ons; each resolves a distinct failure mode of task-only routing.

4.1 Learning Profile-Conditioned Routing

Counterfactual Supervision. Existing skill retrieval datasets such as SkillRet (Cho, Kang, and Kim 2026), Skills-Bench (Li et al. 2026), SWE-Skills (Han et al. 2026), and SRA (Su et al. 2026) provide task-skill pairs, but cannot specify when a profile should overturn an otherwise plausible task-only match. We therefore construct training supervision around task-fixed counterfactual pairs (illustrated in Figure 2), separately from the held-out benchmark. Starting from a target skill S^+ , we generate a task q and a compatible profile p , then mine task-relevant skills that conflict with p through retrieval and verification as hard negatives set S^- . For one such skill $\tilde{S}^+ \in S^-$, we generate a counterfactual

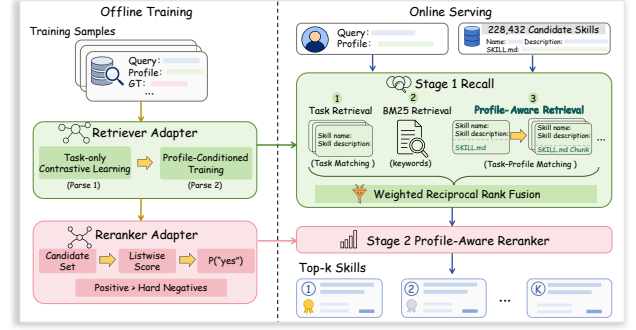


Figure 3: Overview of the proposed personalized skill routing framework. During offline training, user profile-aware supervision is constructed to optimize a dense retriever and a reranker. During online serving, the retriever recalls candidate skills from the repository, and the reranker produces the final personalized ranking.

profile $\tilde{p}$ under which it becomes the preferred skill for the same q and subsequently mine a corresponding set of hard negatives $\tilde{S}^-$. The resulting paired tuples (p, q, S^+, S^-) and $(\tilde{p}, q, \tilde{S}^+, \tilde{S}^-)$ therefore differ only in the routing-relevant profile but require different rankings. They directly supervise the desired reversal: profile information must discriminate between functionally plausible skills, rather than merely accompany task wording. We remove likely valid alternatives, near duplicates, and invalid skill files to limit false negatives; the resulting corpus contains 2,712 task-centric and 7,459 profile-conditioned instances, with examples in Appendix Table 5.

Two-Phase Retriever Training. We adapt Qwen3-Embedding-0.6B and Qwen3-Reranker-0.6B (Zhang et al. 2025) to complementary stages of routing. The retriever operates over the full repository, where a missed functionally relevant skill cannot be recovered, whereas the reranker receives only task-relevant alternatives and can focus on their profile-conditioned differences. Pretrained embeddings capture general semantic similarity rather than skill-specific capability matching. We consequently train the retriever progressively. Phase 1 uses task-only queries and hard negatives to establish routing-specific task-skill alignment. Phase 2 adds profile constraints and the counterfactual pairs above, making the profile a discriminative signal among functionally plausible skills rather than a shortcut for weak task matching. Both phases use InfoNCE (van den Oord, Li, and Vinyals 2019) over the reference skill, curated hard negatives, and in-batch negatives.

Profile-Aware Reranker Training. Unlike the retriever, the reranker is trained directly on profile-conditioned candidate groups. Each group contains one target and task-relevant but profile-conflicting hard negatives; candidates include their names, descriptions, and procedural bodies. We optimize a listwise objective (Xia et al. 2008) using the model’s yes logit as the relevance score. This objective reflects the final routing decision: the suitable skill must out-

rank near misses that satisfy the task in isolation but violate a profile constraint. Complete objectives are provided in Appendix C.2.

4.2 Profile-Aware Skill Routing

Task relevance alone does not establish profile compatibility. Although two skills may expose the same high-level capability, the conditions that make one appropriate for a particular user often differ significantly. For example, required inputs, supported platforms, output formats, dependencies, and exceptions are often stated only in their procedural bodies. SkillFeed therefore performs *profile-aware skill-body retrieval*: it retrieves localized, profile-relevant conditions from skill body for a task–profile query and converts the retrieved chunks into a skill-level compatibility signal. The signal complements, rather than replaces, task relevance: it identifies which task-relevant skills state execution conditions that fit the requesting profile.

Structured Skill-Body Representation. Skill descriptions are concise summaries of core functionality (OpenAI 2026), whereas profile-relevant conditions are distributed across sections such as *When to use*, *Inputs*, *Examples*, *Constraints*, and *Dependencies*. Encoding an entire skill as one document would mix these conditions with unrelated capabilities and dilute a decisive local match. We instead split each file into Markdown-structured chunks, following passage-level retrieval (Lewis et al. 2020), and encode them independently. Each chunk retains its skill name, description, section path, and content; long sections are divided into overlapping passages. This representation turns an implicit document-level judgment into retrievable evidence: a profile constraint can match the precise passage that supports or rules out a skill.

Evidence Aggregation. For a task–profile query, profile-aware skill-body retrieval returns matching chunks, which we aggregate into a skill-level compatibility score. For each candidate skill S_i , let $\mathcal{C}_{S_i}$ denote the set of retrieved chunks belonging to S_i , and let h_c denote the dense similarity score of chunk c . Let $\text{TopM}(\mathcal{C}_{S_i})$ denote the set of the M highest-scoring chunks in $\mathcal{C}_{S_i}$. The aggregated score is computed as

$$\text{score}(S_i) = \alpha \max_{c \in \mathcal{C}_{S_i}} h_c + \frac{\beta}{M} \sum_{c \in \text{TopM}(\mathcal{C}_{S_i})} h_c + \gamma \log(1 + |\mathcal{C}_{S_i}|), \quad (3)$$

where α , β , and γ are non-negative aggregation weights, and M is the number of highest-scoring chunks included in the mean term. The maximum term preserves a single decisive condition, the top- M mean distinguishes repeated evidence from an isolated noisy passage, and the logarithmic count only mildly rewards multiple supporting chunks. This converts local passages into a compatibility signal without mechanically favoring longer skill files.

Integrating Evidence into Routing. Profile-aware skill-body retrieval is combined with one profile signal and two task signals: profile dense over profile, task dense retrieval over skill, and BM25 (Robertson and Zaragoza 2009) over

the same text. The latter preserves exact lexical matches, such as tool names, platforms, and file formats that dense representations may underweight. We combine the three ranked lists with weighted reciprocal rank fusion (Cormack, Clarke, and Buettcher 2009), which avoids calibrating heterogeneous raw scores. The reranker then receives the task, profile, and complete skill information to resolve any remaining conflicts. Thus, task-level retrieval establishes functional plausibility, while profile-aware skill-body retrieval exposes the local conditions needed to choose among plausible skills.

5 Experiment

We evaluate three questions. First, does profile conditioning improve routing precisely when a profile changes the suitable skill? Second, how does SkillFeed compare with general-purpose, skill-specific, and larger pretrained retrieval and reranking models? Third, do its retrieval, reranking, and profile-aware skill-body components make complementary contributions? Unless otherwise stated, all results are evaluated on SkillFeed-Bench.

5.1 Experimental Setup

SkillFeed is initialized from Qwen3-Embedding-0.6B for retrieval and Qwen3-Reranker-0.6B for reranking. We evaluate both a skill-level profile-conditioned dense channel and the proposed profile-aware skill-body retrieval channel; each can be fused with task-oriented dense retrieval and BM25 before reranking. Unless otherwise stated, the system reranks the top 50 fused candidates. Complete training, preprocessing, and inference settings are provided in Appendix C.3.

Evaluation Protocol. We report Hit@ K for $K \in \{1, 3, 5, 10\}$, MRR@20, and nDCG@20. Hit@ K measures whether the annotated skill is surfaced within the top K results, whereas MRR@20 and nDCG@20 emphasize its early-rank position. To evaluate profile sensitivity, we report results both on the full benchmark and on 162 profile-sensitive queries with user context constraints. We also refer to Hit@1 on its 77 profile-counterfactual subset as counterfactual-switch accuracy (CF-Switch Acc.).

Baselines. We compare SkillFeed with retrieval and reranking baselines corresponding to the two stages of our pipeline.

Retrievers. We include BM25 (Robertson and Zaragoza 2009) as a sparse lexical baseline. E5-large-v2 (Wang et al. 2022) and BGE-large-en-v1.5 (Xiao et al. 2024) represent general-purpose dense retrievers. We further evaluate the dense retriever from SkillRouter (Zheng et al. 2026), which is specifically trained for task-to-skill routing. Qwen3-Embedding-8B (Zhang et al. 2025) is included without task-specific adaptation to examine whether model scale alone can replace routing-specific supervision. Finally, we report our Phase 1 retriever to isolate the effect of profile-conditioned adaptation.

Rerankers. We compare our reranker against the pretrained Qwen3-Reranker-0.6B (Zhang et al. 2025), BGE-reranker-v2-m3 (Chen et al. 2024), and the pretrained Qwen3-Reranker-8B (Zhang et al. 2025). We additionally use MiMo-V2.5-Pro (Xiaomi MiMo Team 2026) as LLM reranker,

Method	Input	All Hit@1	Sensitive Hit@1	Insensitive Hit@1	CF-Switch Acc.
Qwen3-Emb-0.6B + Qwen3-Reranker-0.6B	Task + Profile	0.520	0.426	0.611	0.143
SkillFeed Emb Phase 1	Task	0.483	0.340	0.623	0.052
SkillFeed Emb Phase 2	Task + Profile	0.617	0.512	0.719	0.286
SkillFeed Emb Phase 2 + SkillFeed Reranker	Task	0.644	0.444	0.838	0.026
SkillFeed Emb Phase 2 + SkillFeed Reranker	Task + Profile	0.733	0.630	0.832	0.377

Table 1: Effect of profile conditioning at the retrieval and reranking stages. “Task” denotes task-only conditioning, whereas “Task + Profile” denotes profile-aware conditioning. All metrics are computed on 329 test samples, 162 profile-sensitive samples and 167 profile-insensitive queries. CF-Switch Acc. denotes Hit@1 on 77 profile-counterfactual samples.

Embedding Model	Reranker Model	Hit@1	Hit@3	Hit@5	Hit@10	MRR@20	nDCG@20
<i>(1) Embedding Only (No Reranker)</i>							
BM25 Only	—	0.410	0.538	0.578	0.611	0.485	0.528
E5-large-v2	—	0.322	0.474	0.508	0.565	0.410	0.459
BGE-large-en-v1.5	—	0.386	0.492	0.556	0.626	0.461	0.509
Qwen3-Emb-0.6B	—	0.416	0.559	0.644	0.717	0.514	0.575
SkillFeed Emb Phase 1 (Task-only FT)	—	0.483	0.638	0.699	0.787	0.583	0.642
Qwen3-Emb-8B	—	0.471	0.574	0.634	0.724	0.574	0.634
SkillFeed Emb Phase 2 (User profile FT)	—	0.617	0.754	0.796	<u>0.833</u>	0.697	0.741
BM25 + Task Dense + Profile Dense	—	0.492	0.641	0.705	0.815	0.597	0.663
BM25 + Task Dense + Profile Chunk Dense	—	<u>0.517</u>	<u>0.705</u>	<u>0.775</u>	0.860	<u>0.632</u>	<u>0.698</u>
<i>(2) Embedding + Reranker</i>							
SkillFeed Emb Phase 2	Qwen3-Reranker-0.6B	0.559	0.757	0.818	<u>0.882</u>	0.668	0.727
SkillFeed Emb Phase 2	BGE-reranker-v2-m3	0.304	0.495	0.596	0.702	0.434	0.525
SkillFeed Emb Phase 2	Qwen3-Reranker-8B	<u>0.632</u>	<u>0.772</u>	<u>0.821</u>	0.881	<u>0.717</u>	<u>0.763</u>
SkillFeed Emb Phase 2	LLM (mimo-v2.5-pro)	0.590	0.726	0.769	0.824	0.672	0.715
Qwen3-Emb-8B	Qwen3-Reranker-8B	0.602	0.723	0.787	0.836	0.681	0.724
SkillRouter Emb 0.6B	SkillRouter Reranker 0.6B	0.614	0.726	0.751	0.772	0.675	0.703
SkillFeed Emb Phase 2	SkillFeed Reranker	0.733	0.818	0.863	0.891	0.788	0.819
<i>(3) Full Pipeline (BM25 + Dense + Reranker)</i>							
BM25 + Task Dense + Profile Dense	SkillFeed Reranker	0.751	<u>0.836</u>	<u>0.869</u>	<u>0.900</u>	0.803	<u>0.831</u>
BM25 + Task Dense + Profile Chunk Dense	SkillFeed Reranker	<u>0.739</u>	0.851	0.888	0.918	0.803	0.838

Table 2: Main results on SkillFeed-Bench. We organize results by pipeline stage: (1) Embedding Only, (2) Embedding + Reranker, and (3) Full Pipeline. Bold indicates the best result, and underlining indicates the second-best result. Task/Profile Dense use SkillFeed Emb Phase 2 for respective inputs; Profile Chunk Dense chunks input.

prompting it to assess each candidate with respect to both the task requirements and the profile constraints.

All retrievers search the same 228K-skill repository. For reranker comparisons, every model reranks the same fixed candidate lists, so differences can be attributed to candidate discrimination rather than to recall coverage.

5.2 Main Results

Necessity of Personalized Skill Routing. Table 1 presents a controlled comparison between task-only and profile-conditioned routing configurations. The pretrained Qwen baseline and the task-only Phase 1 retriever achieve only 0.143 and 0.052 Hit@1 on profile-counterfactual queries, despite obtaining 0.611 and 0.623 on profile-insensitive queries. This substantial gap demonstrates that task semantics alone are insufficient for personalized skill routing and confirms the need for profile-counterfactual eval-

uation. Incorporating the constructed profile-aware supervision in Phase 2 improves counterfactual Hit@1 from 0.052 to 0.286. Moreover, adding user profile conditioning to the reranker further increases counterfactual Hit@1 from 0.026 to 0.377, while insensitive-query performance changes only marginally from 0.838 to 0.832. These findings demonstrate that personalized skill routing is necessary under profile shifts: the counterfactual construction in SkillFeed-Bench exposes skill changes that task-level evaluation overlooks, while fine-tuning the retriever and reranker on profile-aware data with counterfactual supervision enables them to effectively recognize and resolve these changes.

Overall Comparison. Table 2 compares SkillFeed with representative baselines at three increasingly complete settings: embedding-only retrieval, retrieval followed by reranking, and the full hybrid system. This organization separates the contribution of routing-specific representation learning,

Pipeline Configuration	Hit@1	Hit@5	Hit@10	MRR@20
<i>Single-Stage Retrieval</i>				
BM25 Only	0.410	0.578	0.611	0.485
SkillFeed Emb Phase 2	0.617	0.796	0.833	0.697
<i>Two-Stage: Dense + Reranker</i>				
SkillFeed Emb Phase 2 + Qwen3-Reranker-0.6B	0.559	0.818	0.882	0.668
SkillFeed Emb Phase 2 + SkillFeed Reranker	0.733	0.863	0.891	0.788
<i>Full Pipeline: BM25 + Dense + Reranker</i>				
Full Pipeline (topk=50)	0.751	0.869	0.900	0.803
Full Pipeline (topk=100)	0.739	0.866	0.903	0.796
<i>Ablation: Pipeline Components</i>				
Full Pipeline w/o BM25	0.723	0.878	0.909	0.788
Full Pipeline w/o Profile Dense	0.726	0.854	0.891	0.785

Table 3: Ablation on retrieval pipeline. The full pipeline defaults topk=50. “w/o” indicates specific components are removed.

profile-aware candidate discrimination, and complementary candidate-generation signals.

Profile-Aware Embedding and Reranking. SkillFeed Emb Phase 2 achieves the best embedding-only performance, reaching 0.617 Hit@1, compared with 0.483 for the task-only Phase 1 retriever and 0.471 for the unfine-tuned Qwen3-Embedding-8B model. Other general-purpose baselines perform substantially worse, with BM25, E5-large-v2, and BGE-large-en-v1.5 achieving only 0.410, 0.322, and 0.386 Hit@1, respectively. The adapted 0.6B model also outperforms the pretrained 8B model, indicating that domain-specific supervision can offset model-size differences. For reranking, the SkillFeed reranker raises Hit@1 to 0.733 with the Phase 2 retriever, outperforming the Qwen3-Reranker-8B and SkillRouter pipelines by 10.1 and 11.9 percentage points, respectively. These results show that profile-aware supervision improves both candidate retrieval and fine-grained reranking, providing gains beyond task-only training, general-purpose models, and increased model scale.

Multi-Source Recall and Profile Chunk Retrieval. The full pipeline achieves the best Hit@1 of 0.751 and MRR@20 of 0.803 by combining profile-aware dense retrieval, BM25, and SkillFeed reranking. The Profile chunk-based variant slightly lowers Hit@1 to 0.739 but improves Hit@10 to 0.918 and nDCG@20 to 0.838. This trade-off reflects the division of labor between the two stages: retrieval prioritizes placing the ground-truth skill within the top- K candidate set, whereas reranking determines its final position. Overall, the results validate the complementary benefits of profile-aware training, multi-source recall, and skill-aware chunk retrieval.

5.3 Further Analysis

Retrieval Pipeline Ablation. Table 3 shows that replacing the pretrained Qwen3 reranker with the SkillFeed reranker improves Hit@1 from 0.559 to 0.733, demonstrating the importance of routing-specific reranking. The full pipeline further reaches 0.751 Hit@1 and 0.803 MRR@20, confirming that multi-source recall and profile-aware reranking provide complementary benefits. Removing profile dense retrieval reduces both Hit@1 and MRR@20, validating its role in retrieving persona-relevant candidates. Removing BM25 low-

ers early-rank precision but slightly improves deeper recall, indicating that lexical retrieval provides complementary but rank-sensitive evidence. The top-50 setting offers the best balance between early precision and deeper coverage.

Performance Across User Profile Groups. Appendix Table 6 analyzes the model across country, language, and domain groups. While SkillFeed achieves 0.733 overall Hit@1, its lower performance on specialized technical and scientific queries indicates that fine-grained domain and user profile constraints remain challenging. At the same time, the results demonstrate that the proposed framework generalizes across heterogeneous user profiles rather than being limited to a single user profile category.

Statistical Significance. Tables 7 and 8 in the appendix report bootstrap confidence intervals and paired permutation tests. The full system achieves Hit@1 of 0.751 with a 95% confidence interval of [0.706, 0.794] and improves on the pretrained Qwen baseline by 0.231 ($p < 0.0001$). The gains from the SkillFeed reranker over embedding-only retrieval (+0.116, $p = 0.001$) and from including the profile in reranking (+0.089, $p < 0.01$) are also significant. These tests support the reliability of the central comparisons reported above.

6 Conclusion

This work reframes skill routing in LLM agents as a profile-conditioned relevance problem. We show that task semantics alone cannot reliably identify the correct skill when user constraints induce different routing decisions: on the profile-sensitive queries in SkillFeed-Bench, task-only retrieval achieves only 0.426 Hit@1, while personalized retrievers raise it to 0.630. SkillFeed, our two-stage framework combining multi-source candidate recall with profile-conditioned reranking, achieves 0.751 Hit@1 overall, with the largest gains concentrated on profile-sensitive instances. These results establish user profile information as an essential signal for personalized skill routing in agentic systems. Future work will explore dynamically extracting and updating user constraints from multi-turn conversational contexts (Liu et al. 2026).

References

- Bang, S.; and Song, H. 2025. Llm-based user profile management for recommender system. *arXiv preprint arXiv:2502.14541*.
- Chen, J.; Xiao, S.; Zhang, P.; Luo, K.; Lian, D.; and Liu, Z. 2024. Bge m3-embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 4(5).
- Cho, H.; Kang, R.; and Kim, Y. 2026. SkillRet: A Large-Scale Benchmark for Skill Retrieval in LLM Agents. *arXiv:2605.05726*.
- Cormack, G. V.; Clarke, C. L. A.; and Buettcher, S. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 758–759.
- Deldjoo, Y.; He, Z.; McAuley, J.; Korikov, A.; Sanner, S.; Ramisa, A.; Vidal, R.; Sathiamoorthy, M.; Kasirzadeh, A.; and Milano, S. 2024. A Review of Modern Recommender Systems Using Generative Models (Gen-RecSys). *arXiv:2404.00579*.
- Han, T.; Zhang, Y.; Song, W.; Fang, C.; Chen, Z.; Sun, Y.; and Hu, L. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? *arXiv:2603.15401*.
- Koren, Y.; Bell, R.; and Volinsky, C. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer*, 42(8): 30–37.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.
- Li, F.; Tagkopoulou, P.; and Tagkopoulou, I. 2026. SkillFlow: Scalable and Efficient Agent Skill Retrieval System. In *Deep Learning for Code: Towards Human-Centered Coding Agents*.
- Li, X.; Liu, Y.; Chen, W.; You, B.; Di, Z.; He, Y.; Zheng, S.; Choe, K. W.; Sun, J.; Wang, S.; Tao, C.; Li, B.; Zhao, X.; Geng, H.; Wu, X.; Zhou, J.; Chen, X.; Xing, H.; Li, Y.; Zeng, Q.; Wang, D.; Wang, Y.; Chaim, R. B.; Jiang, P.; Shen, H.; Kong, L.; Liu, X.; Wang, R.; Liu, X.; Li, J.; Lan, X.; Lin, Y.; Ye, W.; He, J.; Li, S.; Zhang, Y.; Gao, Y.; Li, Y.; Ma, Z.; Jing, L.; Wang, T.; Li, K.; Xue, Y.; Lyu, H.; He, Y.; Tian, Y.; Wu, S.; Wang, B.; Gao, Y.; Chen, B.; Liu, L.; Cheng, S.; Bao, J.; Tong, S.; Xu, S.; Zhuo, T. Y.; Ye, T.; Qi, Q.; Li, M.; Liao, L.; Tan, Z.; Shi, C.; Tang, X.; Tankasala, S.; Yuan, B.; Qian, Y.; Tu, J.; Wang, C.; Sun, Y.; Wang, W.; Taylor, A.; Yang, Z.; Guan, C.; Dong, Z.; Zhang, X.; Dillmann, S.; chung Lee, H.; and Song, D. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv:2602.12670*.
- Liang, Y.; Zhong, R.; Xu, H.; Jiang, C.; Zhong, Y.; Fang, R.; Gu, J.-C.; Deng, S.; Yao, Y.; Wang, M.; Qiao, S.; Xu, X.; Wu, T.; Wang, K.; Liu, Y.; Bi, Z.; Lou, J.; Jiang, Y. E.; Zhu, H.; Yu, G.; Hong, H.; Huang, L.; Xue, H.; Wang, C.; Wang, Y.; Shan, Z.; Chen, X.; Tu, Z.; Xiong, F.; Xie, X.; Zhang, P.; Gui, Z.; Liang, L.; Zhou, J.; Wu, C.; Shang, J.; Gong, Y.; Lin, J.; Xu, C.; Deng, H.; Zhang, W.; Ding, K.; Zhang, Q.; Huang, F.; Zhang, N.; Pan, J. Z.; Qi, G.; Wang, H.; and Chen, H. 2026. SkillNet: Create, Evaluate, and Connect AI Skills. *arXiv:2603.04448*.
- Liu, H.; Sun, Z.; Wei, T.; Wang, Y.; Zhu, J.; and Qu, X. 2026. Diagnostic-guided dynamic profile optimization for llm-based user simulators in sequential recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 15306–15314.
- OpenAI. 2026. Skills. <https://developers.openai.com/api/docs/guides/tools-skills>. Accessed: 2026-07-05.
- Rendle, S.; Freudenthaler, C.; Gantner, Z.; and Schmidt-Thieme, L. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence*, 452–461.
- Robertson, S.; and Zaragoza, H. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4): 333–389.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2024. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*, 36.
- Shen, Y.; Song, K.; Tan, X.; Li, D.; Lu, W.; and Zhuang, Y. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. *arXiv preprint arXiv:2303.17580*.
- SkillsMP. 2026. Agent Skills Marketplace. <https://skillsmp.com>. Accessed: 2026-07-26.
- Su, W.; Long, J.; Ai, Q.; He, Q.; Tang, Y.; Wang, C.; Tu, Y.; Wang, Y.; and Liu, Y. 2026. Skill Retrieval Augmentation for Agentic AI. *arXiv:2604.24594*.
- van den Oord, A.; Li, Y.; and Vinyals, O. 2019. Representation Learning with Contrastive Predictive Coding. *arXiv:1807.03748*.
- Wang, L.; Yang, N.; Huang, X.; Jiao, B.; Yang, L.; Jiang, D.; Majumder, R.; and Wei, F. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-Training. *arXiv preprint arXiv:2212.03533*.
- Wang, Z.; Wen, W.; Ji, Q.; Qiao, R.; and Sun, X. 2026. Skill Is Not Document: A Query-Conditional Benchmark and Two-Stage Retriever for LLM Agent Skill Routing. *arXiv preprint arXiv:2606.03565*.
- Westhäußer, R.; Minker, W.; and Zepf, S. 2025. Enabling Personalized Long-term Interactions in LLM-based Agents through Persistent Memory and User Profiles. *arXiv:2510.07925*.
- Xia, F.; Liu, T.-Y.; Wang, J.; Zhang, W.; and Li, H. 2008. Listwise approach to learning to rank: theory and algorithm. In *Proceedings of the 25th International Conference on Machine Learning, ICML '08*, 1192–1199. New York, NY, USA: Association for Computing Machinery. ISBN 9781605582054.
- Xiao, S.; Liu, Z.; Zhang, P.; Muennighoff, N.; Lian, D.; and Nie, J.-Y. 2024. C-Pack: Packed resources for general chinese

embeddings. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*, 641–649.

Xiaomi MiMo Team. 2026. MiMo-V2.5-Pro. <https://huggingface.co/collections/XiaomiMiMo/mimo-v25>.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*.

Zhang, Y.; Li, M.; Long, D.; Zhang, X.; Lin, H.; Yang, B.; Xie, P.; Yang, A.; Liu, D.; Lin, J.; Huang, F.; and Zhou, J. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176*.

Zhang, Z.; Shi, K.; Huang, S.; Nie, A.; Zeng, Y.; Zhao, Y.; Fang, Z.; Su, Q.; Qiu, H.; Yang, W.; Ren, Q.; Zou, S.; Huang, W.; Chen, L.; Chen, Z.; and Zhao, F. 2026. SkillFlow: Benchmarking Lifelong Skill Discovery and Evolution for Autonomous Agents. *arXiv preprint arXiv:2604.17308*.

Zheng, Y.; Zhang, Z.; Ma, C.; Yu, Y.; Zhu, J.; Wu, Y.; Xu, T.; Dong, B.; Zhu, H.; Huang, R.; and Yu, G. 2026. SkillRouter: Skill Routing for LLM Agents at Scale. *arXiv preprint arXiv:2603.22455*.

Zhong, S.; Lu, Y.; Ning, J.; Wan, Y.; Feng, L.; Ao, Y.; Ribeiro, L. F. R.; Dreyer, M.; Ammirati, S.; and Xiong, C. 2026. SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks. *arXiv preprint arXiv:2604.20087*.

Zhou, Y.; Zhang, Z.; Cheng, Z.; Lan, Q.; Chen, Z.; Yang, Z.; Xu, Q.; Chen, R.; Wang, H.; and Hu, S. 2026. SkillGenBench: Benchmarking Skill Generation Pipelines for LLM Agents. *arXiv preprint arXiv:2605.18693*.

Zhou, Y.; Zhu, Q.; Jin, J.; and Dou, Z. 2024. Cognitive Personalized Search Integrating Large Language Models with an Efficient Memory Mechanism. *arXiv:2402.10548*.

A benchmark

A.1 Test Sample Format

Each test sample in SkillFeed-Bench is a JSON line in `test.jsonl`. The top-level structure is shown below:

```
{
  "task": {
    "name": "...",
    "description": "...",
    "core_requirements": ["...", "..."]
  },
  "user_context_constraints": {
    "country_region": "...",
    "language": "...",
    "preferred_platforms": ["..."],
    "payment_methods": ["..."],
    "other_constraints": ["..."]
  },
  "ground_truth_skill": {...},
}
```

The `task` object describes the user’s natural-language request; `user_context_constraints` captures the profile attributes (country, language, platform, payment, domain constraints); `ground_truth_skill` identifies the single correct skill from the 228K candidate pool. The 329 test samples include 162 profile-sensitive queries (77 profile-counterfactual augmentation) and 167 profile-insensitive queries.

Each retained original and counterfactual instance was independently reviewed by two annotators with prior experience in AI agents, software tools, and programming-related tasks. We provided annotation guidelines specifying three criteria: (1) whether the skill is capable of completing the task, (2) whether user profile information changes the preferred skill, and (3) whether the counterfactual profile preserves the original task while introducing a meaningful preference shift. Disagreements were resolved by an expert reviewer using the same criteria, and ambiguous cases without clear consensus were discarded. This verification process ensures that benchmark labels reflect profile-conditioned skill suitability rather than superficial lexical overlap.

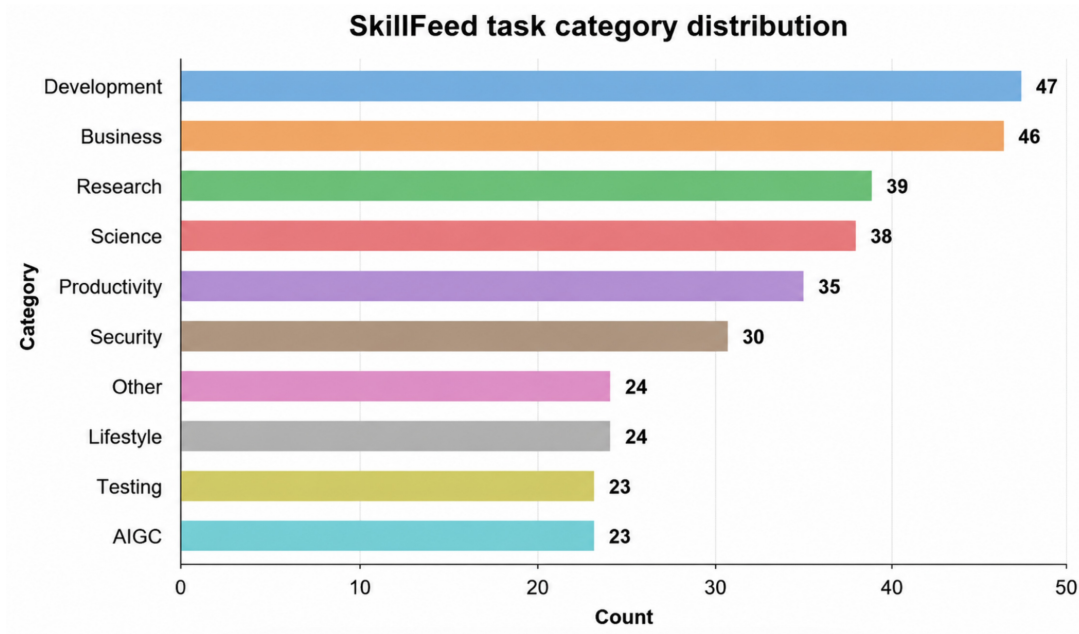


Figure 4: SkillFeed-Bench consists of tasks spanning 10 categories.

Table 4 summarizes the key statistics. The test set consists of 329 samples drawn from a candidate pool of 228,432 skills. Each sample is annotated with 4 hard negative skills that are semantically similar but profile-incompatible. The average task description is 61.8 words, and the average skill body excerpt is 16,730 characters. To prevent trivial lexical matching, we enforce a token overlap threshold of ≤ 0.50 between the task description and the ground-truth skill body, with an average overlap of 0.29. The annotator confidence averages 0.92, indicating high annotation quality.

Table 4: Key statistics of SkillFeed-Bench.

Statistic	Value
Total skill candidates	228,432
Test samples	329
– Profile-sensitive	162
– Profile counterfactual	77
– Standard (with constraints)	85
– Profile-insensitive	167
Unique skill categories	10
Avg. task description length (words)	61.8
Avg. skill body excerpt length (chars)	16,730
Token overlap (task–skill body)	0.29 (max 0.50)
Annotator confidence (avg.)	0.92

The test set covers 10 skill categories spanning a diverse range of domains. As shown in Table 4, the two largest categories are Development (47 samples, 14.3%) and Business (46 samples, 14.0%), followed by Research (39, 11.9%) and Science (38, 11.6%). The remaining six categories—Productivity, Security, Other, Lifestyle, Testing, and AIGC—each contribute 23–35 samples (7.0%–10.6%), ensuring a balanced coverage across application domains.

B Training and Pipeline

B.1 Learning Profile-Conditioned Routing

User Profile-Aware Supervision Construction. Existing skill retrieval datasets such as SkillRet (Cho, Kang, and Kim 2026), SkillsBench (Li et al. 2026), SWE-Skills (Han et al. 2026), and SRA (Su et al. 2026) are valuable for general skill retrieval, but they are insufficient for modeling personalized skill routing. To address this limitation, as illustrated in the right-hand flow of Figure 2, we construct a training dataset that incorporates both profile-based and counterfactual supervision.

Starting from a target skill S^+ , an LLM generates a realistic task query q and a compatible profile p based on the skill description and procedural content. The profile captures routing-relevant constraints, such as language, region, preferred platform, and payment method. We then construct a hard-negative set S^- by combining semantic retrieval with LLM-based verification, prioritizing skills that are relevant to the task but incompatible with the user profile. This process yields task-centric supervision tuples (p, q, S^+, S^-) .

To strengthen user profile-conditioned supervision, we further perform target-first counterfactual augmentation. Specifically, a hard negative $\tilde{S}^+ \in S^-$ is selected as the counterfactual target, and an LLM generates a new profile $\tilde{p}$ under which $\tilde{S}^+$ is preferred for the same query, yielding $(\tilde{p}, q, \tilde{S}^+, S^-)$. These paired instances provide direct supervision for learning how profile changes affect skill preference while the task remains unchanged. Finally, we filter likely valid alternatives, near-duplicate skills, and candidates without valid skill content to reduce false negatives. The resulting dataset contains 2,712 task-centric and 7,459 user profile-conditioned instances, with representative examples shown in Appendix Table 5.

Two-Phase Retriever Training. Using the constructed supervision dataset, we adapt Qwen3-Embedding-0.6B and Qwen3-Reranker-0.6B (Zhang et al. 2025) to their complementary roles in personalized skill routing. The dense retriever must preserve broad task–skill relevance while incorporating user profile information into candidate recall, whereas the reranker focuses on fine-grained user profile-conditioned discrimination among semantically relevant candidates.

We optimize the retriever in two phases. Although the pretrained embedding model captures general semantic similarity, it is not specialized for skill routing, which requires matching task requirements to the capabilities and applicability conditions encoded in skill documents. Phase 1 therefore uses task-only queries and hard negatives to learn routing-specific task–skill alignment. Phase 2 appends profile constraints to the query and introduces paired instances in which the same task is associated with different reference skills. This schedule first adapts the model to skill routing and then learns user profile-conditioned discrimination.

For each training instance, let u_i denote the query representation, S_i^+ the reference skill, and $\mathcal{N}_i$ the set of curated and in-batch negative skills. In Phase 1, the query contains only the task, i.e., $u_i = q_i$. In Phase 2, the user profile is appended to the task, i.e., $u_i = [q_i; p_i]$.

	Phase 1: Task-Only Training Data	Phase 2: Profile-Conditioned Training Data
Task	I want to plan a trip to Shanghai including attractions and a simple schedule.	
User Profile	(Not included in query)	Occupation: Student Budget: Low budget Transportation: Subway preferred
Query Format	Task name: Shanghai Travel Planning Task description: Plan a trip to Shanghai including attractions and a simple itinerary.	Task name: Shanghai Travel Planning Task description: Plan a trip to Shanghai including attractions and a simple itinerary. User/context constraints: occupation: Student budget: Low budget transportation: Subway preferred
Ground Truth	city-travel-guide	budget-travel-planner
Why GT is correct	The query only specifies the travel planning task, so a general-purpose travel planning skill is sufficient.	The user profile indicates limited budget and preference for public transportation, making a budget-oriented planner the most suitable skill.
Hard Negative	attraction-recommender	luxury-itinerary-planner
Why HN is wrong	The skill focuses only on attraction recommendation and cannot generate a complete travel itinerary.	Assumes premium travel preferences and taxi transportation, which conflict with the user profile.

Table 5: Training data examples for two-stage skill retrieval. Phase 1 learns general task-to-skill matching using only the task description, while Phase 2 incorporates user profile constraints to enable personalized skill selection.

We optimize the dense retriever with the InfoNCE objective (van den Oord, Li, and Vinyals 2019):

$$\mathcal{L}_{\text{ret}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\text{sim}(E_{\theta}(u_i), E_{\theta}(S_i^+)) / \tau_{\text{ret}})}{\sum_{S \in \{S_i^+\} \cup \mathcal{N}_i} \exp(\text{sim}(E_{\theta}(u_i), E_{\theta}(S)) / \tau_{\text{ret}})}, \quad (4)$$

where E_{θ} is the dense encoder, $\text{sim}(\cdot, \cdot)$ denotes cosine similarity, B is the batch size, and τ_{ret} is the temperature. The objective encourages the reference skill to be ranked above skills that are task-relevant but incompatible with the user profile.

Persona-Aware Reranker Training. Unlike the dense retriever, the reranker is trained directly with user profile-aware supervision because it operates on a small candidate set whose members are already relevant to the task. Its primary role is therefore to determine which candidate best satisfies the user profile-specific constraints. For each query containing both task and profile information, we construct a ranking group consisting of one positive skill and multiple curated hard negatives. Each candidate is represented by its name, description, and procedural body.

The reranker operates on a candidate group containing one reference skill S_i^+ and a set of hard negatives $\mathcal{N}_i$. Given query u_i and candidate skill S , Qwen3-Reranker produces a relevance score $r_{\phi}(u_i, S)$ from the logit assigned to the token *yes*. We optimize the model using listwise cross-entropy:

$$\mathcal{L}_{\text{rank}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(r_{\phi}(u_i, S_i^+) / \tau_{\text{rank}})}{\sum_{S \in \{S_i^+\} \cup \mathcal{N}_i} \exp(r_{\phi}(u_i, S) / \tau_{\text{rank}})}, \quad (5)$$

where τ_{rank} is the ranking temperature. Normalizing scores within each candidate group directly trains the reference skill to outrank semantically similar alternatives.

B.2 Persona-Aware Routing Pipeline

At inference time, SkillFeed adopts a two-stage inference pipeline that separates broad candidate coverage from fine-grained candidate discrimination. The first stage retrieves candidate skills from the 228K-scale repository, and the second stage reranks the retrieved candidates. This separation allows the recall stage to focus on preserving potentially relevant skills, while the reranker determines which candidate best satisfies both the task requirement and the user profile.

Candidate Recall. The recall stage combines three complementary retrieval channels. The first channel is task-oriented dense retrieval, which encodes the task name and task description as the query and retrieves skills according to their names and descriptions. The second channel applies BM25 over the same skill-level text, providing a lexical matching signal complementary to dense retrieval (Robertson and Zaragoza 2009). The third channel is persona-conditioned dense retrieval over structured chunks

extracted from `SKILL.md` bodies. Unlike the first two channels, which rely on skill-level summaries, this channel exposes body-level evidence during recall and is designed to capture fine-grained user profile-related information.

Skill descriptions are usually concise and mainly summarize the core function of a skill (OpenAI 2026). However, whether a skill is suitable for a particular user often depends on details that may appear only in the skill body, such as applicable scenarios, input requirements, output formats, tool dependencies, constraints, and examples. These details are commonly distributed across sections such as *When to use*, *Inputs*, *Examples*, *Constraints*, and *Dependencies*. Therefore, relying only on skill names and descriptions may miss candidates whose main function is relevant but whose user profile-specific applicability is expressed in the body text.

Skill-to-Chunk Representation. A straightforward alternative is to embed the entire `SKILL.md` file as a single document. However, this may produce overly coarse representations, since long-document embeddings tend to mix multiple capabilities, conditions, and constraints. As a result, localized but important matching signals may be diluted. Following passage-level retrieval in retrieval-augmented generation (Lewis et al. 2020), we instead split each `SKILL.md` file into Markdown-structured chunks and encode these shorter passages independently.

For each chunk, we store the skill name, description, section path, and content. Long sections are further divided into overlapping passages, so that local applicability conditions can be recovered during recall. This chunk-level index enables profile-conditioned retrieval to match user preferences and constraints against specific skill-body evidence, rather than only against high-level skill summaries.

Chunk-to-Skill Aggregation. User profile-conditioned retrieval returns chunk-level results, which are then aggregated into skill-level candidates before fusion. For each candidate skill S_i , let $\mathcal{C}_{S_i}$ denote the set of retrieved chunks belonging to S_i , and let h_c denote the dense similarity score of chunk c . Let $\text{TopM}(\mathcal{C}_{S_i})$ denote the set of the M highest-scoring chunks in $\mathcal{C}_{S_i}$. The aggregated score is computed as

$$\text{score}(S_i) = \alpha \max_{c \in \mathcal{C}_{S_i}} h_c + \frac{\beta}{M} \sum_{c \in \text{TopM}(\mathcal{C}_{S_i})} h_c + \gamma \log(1 + |\mathcal{C}_{S_i}|), \quad (6)$$

where α , β , and γ are non-negative aggregation weights, and M is the number of highest-scoring chunks included in the mean term. The maximum term captures the strongest local match between the user profile and a passage in the skill body. The top- M mean term measures whether the skill is supported by multiple highly relevant chunks, thereby reducing the influence of an isolated noisy match. The logarithmic count term provides a mild reward for skills associated with multiple retrieved chunks. Overall, this aggregation favors skills with consistent body-level evidence while preventing an excessive preference for skills containing a larger number of chunks.

Fusion and Profile-Aware Reranking. The three recall lists are combined using weighted reciprocal rank fusion (Cormack, Clarke, and Buettcher 2009). The fused top candidates are then passed to the trained reranker, which takes the task, the user profile, and the full candidate skill information as input. In this pipeline, chunk-based retrieval is used to improve user profile-aware candidate coverage during recall. It does not directly determine the final ranking; instead, it increases the likelihood that skills with relevant body-level evidence are included for subsequent fine-grained reranking.

C Additional Experimental Details

C.1 Dataset Structure and Input Formatting

Underlying Sample Schema. Each source instance contains a task, an optional user profile, one reference skill, and a set of hard-negative skills:

```
{
  "task": {
    "name": "...",
    "description": "...",
    "core_requirements": ["...", "..."]
  },
  "user_context_constraints": {
    "country_region": "...",
    "language": "...",
    "preferred_platforms": ["..."],
    "payment_methods": ["..."],
    "other_constraints": ["..."]
  },
  "ground_truth_skill": {...},
  "hard_negative_skills": [{...}, {...}]
}
```

Each skill record contains a name, description, and a `SKILL.md` body excerpt. The reference skill is treated as the single annotated positive for training and evaluation.

Phase 1: Task-Only Supervision. Phase 1 removes the user profile from the query and uses task-centric supervision. The query instruction is:

Represent this user task for retrieving the most appropriate skill.

The resulting query is formatted as:

Represent this user task for retrieving the most appropriate skill.

Task name: {task_name}

Task description: {task_description}

Core requirements:

- {requirement_1}
- {requirement_2}

The training record follows the ms-swift embedding schema:

```
{
  "messages": [{"role": "user", "content": "<query>"}],
  "positive_messages": [
    [{"role": "user", "content": "<positive_skill>"}]
  ],
  "negative_messages": [
    [{"role": "user", "content": "<negative_skill_1>"}],
    [{"role": "user", "content": "<negative_skill_2>"}]
  ]
}
```

Phase 1 contains 2,712 training instances and 356 validation instances.

Phase 2: Profile-Conditioned Supervision. Phase 2 retains the same task fields and appends user-context constraints:

Represent this user task for retrieving the most appropriate skill.

Task name: {task_name}

Task description: {task_description}

Core requirements:

- {requirement_1}
- {requirement_2}

User/context constraints:

country_region: {region}
language: {language}
preferred_platforms: {platforms}
payment_methods: {payment_methods}
other_constraints: {constraints}

Its negatives combine curated hard negatives, LLM-guided profile-conflicting negatives, and counterfactual positives from other profile variants of the same task. The Phase 2 corpus contains 7,459 training instances and 608 validation instances, including full-body and profile-counterfactual supervision.

Skill Document Format. Positive and negative skills are serialized with the same field order:

Skill name: {skill_name}

Skill description: {skill_description}

Skill body:

{skill_body_excerpt}

The body excerpt exposes procedural details and applicability constraints that are not always present in the short description.

Reranker Records. Reranker training uses the task-profile query together with a candidate group containing one positive and multiple negatives. The generative reranker record is:

```
{
```

```

"system":
  "Judge whether the Document meets the requirements based
  on the Query and the Instruct provided. Note that the
  answer can only be \"yes\" or \"no\".",
"messages": [{"role": "user", "content": "<query>"}],
"positive_messages": [
  [{"role": "assistant", "content": "<positive_skill>"}]
],
"negative_messages": [
  [{"role": "assistant", "content": "<negative_skill_1>"}],
  [{"role": "assistant", "content": "<negative_skill_2>"}]
]
}

```

The document role is `user` for embedding records and `assistant` for generative reranker records. During reranker training and inference, the system instruction, query template, skill field names, and constraint serialization are kept consistent to avoid formatting artifacts.

C.2 Training Objectives

C.3 Implementation Details

The SkillFeed dense retriever is initialized from Qwen3-Embedding-0.6B, and the SkillFeed reranker is initialized from Qwen3-Reranker-0.6B. Both models are fine-tuned on $2 \times A100$ GPUs using AdamW with a learning rate of 1×10^{-5} , an effective batch size of 16, a maximum sequence length of 2048, and three training epochs. The retriever is trained in two phases: 2,712 task-centric instances are used for task-only alignment, followed by 7,459 user profile-conditioned instances for profile-aware adaptation. Lexical-overlap filtering uses a threshold of 0.50.

For `SKILL.md` chunk retrieval, Markdown sections are divided into passages with a target length of 200–500 tokens, a maximum length of 800 tokens, and a small overlap between adjacent passages. Chunk-to-skill aggregation uses $\alpha = 0.70$, $\beta = 0.25$, $\gamma = 0.05$, and $M = 3$. Weighted reciprocal rank fusion assigns weights of 0.40 to task dense retrieval, 0.40 to profile-conditioned dense retrieval over `SKILL.md` chunks, and 0.20 to BM25. Unless otherwise stated, the full pipeline reranks the top 50 fused candidates; the top-100 setting is evaluated only in the ablation study.

Table 6: Per-Profile-group analysis on SkillFeed Emb Phase 2 + SkillFeed Reranker with Profile. We evaluate performance across different persona groups: country/region, language, domain, and persona sensitivity. All 329 test samples are included.

Group	N	Hit@1	Hit@5	MRR@20	nDCG@20
<i>By Country/Region</i>					
Global/Unknown	257	0.735	0.879	0.795	0.827
United States	46	0.717	0.804	0.760	0.785
China	14	0.571	0.714	0.648	0.696
Other Regions	12	0.917	0.917	0.917	0.917
<i>By Language</i>					
English	204	0.672	0.804	0.735	0.770
Unknown	100	0.870	0.980	0.907	0.927
Programming	11	0.545	0.818	0.632	0.677
Other Languages	14	0.786	0.929	0.845	0.866
<i>By Domain</i>					
General	248	0.790	0.895	0.836	0.860
Tech	44	0.614	0.818	0.692	0.738
Science	15	0.533	0.800	0.663	0.727
Other Domains	22	0.455	0.682	0.530	0.577
<i>Overall</i>					
Overall	329	0.733	0.863	0.788	0.819

Table 7: Bootstrap 95% confidence intervals for key configurations.

Configuration	Hit@1	95% CI	MRR@20	95% CI
Full Pipeline	0.751	[0.706, 0.794]	0.803	[0.760, 0.846]
SkillFeed Emb Phase 2 + SkillFeed Reranker w Profile	0.733	[0.684, 0.778]	0.788	[0.744, 0.832]
SkillFeed Emb Phase 2 + SkillFeed Reranker w/o Profile	0.644	[0.596, 0.696]	0.712	[0.663, 0.761]
SkillFeed Emb Phase 2	0.617	[0.568, 0.669]	0.697	[0.647, 0.746]
SkillFeed Emb Phase 1	0.483	[0.429, 0.535]	0.583	[0.530, 0.636]

Table 8: Permutation test results for key comparisons. Statistical significance: $*p < 0.05$, $**p < 0.01$, and $***p < 0.001$.

Comparison	Hit@1 Δ	p -value	Sig.
Full Pipeline vs Qwen3-Emb-0.6B + Qwen3-Reranker-0.6B	+0.231	< 0.0001	***
Embedding Only vs Embedding + SkillFeed Reranker	+0.116	0.001	***
Reranker without vs with Profile	+0.089	< 0.01	**